INFORME TÉCNICO: Propuesta de Arquitectura y Flujo de Datos para SGRI-UNICAFAM

Duración del proyecto: 6 meses

1. Resumen Ejecutivo

El presente documento define la arquitectura técnica y el flujo de datos del **Sistema de Gestión de la Investigación Institucional (SGRI-UNICAFAM)**, alineado con el diagrama sistémico actualizado.

La solución se despliega en **Microsoft Azure** y se compone de:

- **Frontend** en React JS
- **Backend** en FastAPI (Python)
- **Base de datos** PostgreSQL con esquemas separados (Profesional, Formativo, Auditoría)
- **Almacenamiento** de documentos en Azure Storage
- **Automatización** de alertas mediante tareas programadas en Azure

El modelo de seguridad utiliza **JWT** para autenticación y **RBAC** con tres grupos de roles: **Administrativos**, **Formativos** (profesores y monitores) y **Estudiantes**.

Los gráficos y dashboards son exclusivos para profesores y administrativos, y su contenido cambia según el rol. Los estudiantes no tienen acceso a gráficos.

Se implementa auditoría completa de cambios y un sistema de alertas unificado por deadlines (proyectos próximos a vencer y semilleros inactivos).

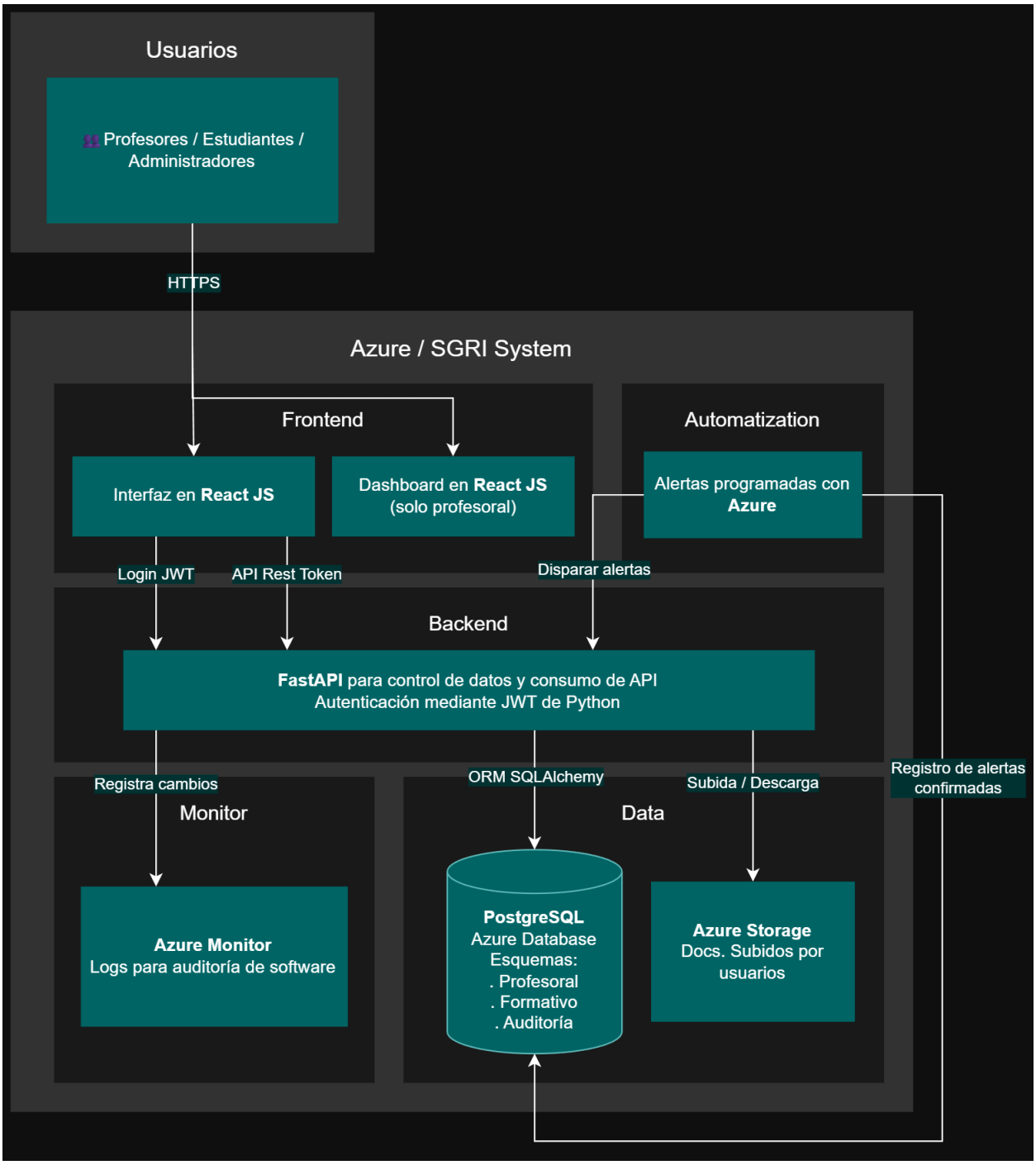
---

2. Alineación con los Requerimientos (MVP)

La siguiente tabla resume la correspondencia entre los requerimientos del SGRI y la implementación propuesta para el MVP. *(Formato apto para presentación)*

Requerimiento del SGRI	Implementación Propuesta (MVP) – Versión resumida
Dato – Dueño – Calidad	Auditoría completa por entidad: UUID, timestamps, usuario responsable, versión, estado y log de cambios.
Roles RBAC	Tres grupos: **Administrativos** / **Formativos** (profesores + monitores) / **Estudiantes**. Control con JWT.
Indicadores y Dashboards	Acceso exclusivo para **profesores y administrativos**. Los gráficos cambian según el rol: profesoral vs administrativo. Estudiantes sin acceso a gráficos.
Alertas Automáticas	Deadlines unificados: proyectos próximos a vencer (30/15/5 días) + semilleros inactivos (>30 días). Alertas programadas en Azure.
Observabilidad y Auditoría	Registro de cambios con SQLAlchemy → tabla auditoría en PostgreSQL (esquema `Auditoría`). Logs centralizados.

3. Arquitectura Técnica de Alto Nivel (MVP)



3.1 Componentes Principales

Componente	Tecnología	Función en el MVP
Frontend	React JS	Interfaz de usuario, login JWT, dashboards por rol

Componente	Tecnología	Función en el MVP
		(solo profesores y administrativos).
Backend	FastAPI (Python)	Autenticación JWT, lógica de negocio, API REST, control de datos.
ORM	SQLAlchemy	Mapeo objeto-relacional, registro automático de cambios hacia auditoría.
Base de datos	PostgreSQL (Azure)	Esquemas separados: `Profesional`, `Formativo`, `Auditoría`.
Almacenamiento	Azure Storage	Documentos subidos por usuarios (planes, informes, actas).
Automatización	Azure Logic Apps / Functions	Ejecución programada diaria para revisar deadlines y enviar alertas.

3.2 Descripción de la Arquitectura

El sistema sigue una arquitectura de capas bien definida:

- **Capa de presentación**: React JS consume la API REST del backend. Se implementan dashboards **solo para profesores y administrativos**:
  - *Dashboard profesoral*: muestra gráficos de avance de sus propios proyectos, alertas de entregas, participación de estudiantes y ejecución presupuestal.
  - *Dashboard administrativo*: muestra gráficos agregados por facultad, monitoreo de indicadores institucionales, rankings y alertas por facultad.
  - *Estudiantes*: solo tienen acceso a formularios y consultas de texto, sin ningún tipo de gráfico o dashboard visual.
- **Capa de lógica de negocio**: FastAPI valida tokens JWT, aplica reglas de negocio (cálculo de avances, verificación de permisos por rol) y registra cada cambio significativo en la tabla de auditoría mediante SQLAlchemy.

- **Capa de datos**: PostgreSQL organiza la información en tres esquemas:
  - `Profesional`: proyectos de investigación, productos, presupuestos.
  - `Formativo`: semilleros, proyectos formativos, eventos, movilidad.
  - `Auditoría`: log de cambios con usuario, entidad, campo, valor anterior/nuevo, timestamp.
- **Almacenamiento externo**: Azure Storage guarda los documentos adjuntos (PDF, Excel, imágenes) y se referencian por URL en la base de datos.
- **Automatización**: Un proceso programado (Azure Logic Apps o Functions) se ejecuta diariamente, consulta la base de datos para detectar deadlines (proyectos a 30/15/5 días de vencer, semilleros inactivos por más de 30 días) y envía notificaciones por correo electrónico a los responsables.

---

4. Flujo de Datos (Ejemplo: Creación de un Proyecto)

El siguiente flujo ilustra la interacción entre componentes para un caso típico:

1. **Inicio de sesión**

   El usuario envía credenciales a `/auth/token`. FastAPI valida y devuelve un token JWT.

2. **Almacenamiento del token**

   El frontend guarda el token y lo incluye en cada petición posterior (cabecera `Authorization: Bearer <token>`).

3. **Envío de datos del nuevo proyecto**

   El usuario (profesor o administrativo) completa el formulario y envía los datos a `/api/proyectos`.

   El backend valida el token y verifica los permisos según el grupo de rol (administrativo o formativo). Los estudiantes no pueden crear proyectos.

4. **Persistencia y auditoría**

   - Se inserta el proyecto en la tabla correspondiente (esquema `Profesional` o `Formativo`).

- SQLAlchemy registra automáticamente la acción en la tabla de auditoría (`Auditoría.cambios`), guardando: `usuario`, `entidad afectada`, `campo modificado`, `valor anterior`, `valor nuevo`, `timestamp`.

5. **Gestión de archivos adjuntos**

   Si el proyecto incluye documentos, se suben a Azure Storage y se almacena la URL pública o firmada en el registro del proyecto.

6. **Respuesta al frontend**

   El backend responde con un código de éxito y el frontend actualiza el dashboard del usuario (si es profesor o administrativo).

7. **Proceso asíncrono de alertas** (ejecutado diariamente)

   Azure Logic Apps consulta la base de datos para identificar:

   - Proyectos cuya fecha de cierre esté a 30, 15 o 5 días.
   - Semilleros sin actividad registrada en los últimos 30 días.
     Para cada caso, envía un correo electrónico al profesor/tutor responsable y, si aplica, al administrativo de la facultad.

---

5. Roadmap de Desarrollo (Duración total: 6 meses)

El desarrollo se organiza en cuatro fases, con entregas incrementales y funcionalidad operativa desde la Fase 2.

Fase	Mes	Actividades Principales	Componentes Implicados
Fase 1 - Fundamentos	Mes 1	Modelo de datos con atributos obligatorios (UUID, timestamps, estado). Esquemas separados (Profesional, Formativo, Auditoría). Autenticación JWT. CRUD básico de usuarios, facultades y roles (tres grupos).	PostgreSQL, FastAPI, frontend base
Fase 2 - Núcleo Operativo	Meses 2-3	Gestión de proyectos (profesoral y formativo). Vinculación de estudiantes a semilleros. Dashboards simples	Ampliación de esquemas, endpoints

Fase	Mes	Actividades Principales	Componentes Implicados
		solo para profesores y administrativos (sin gráficos complejos aún). Registro de eventos académicos.	específicos, React JS
Fase 3 - Gráficos y Alertas	Meses 4-5	Implementación completa de **gráficos por rol** (barras, líneas, donut, calendario, etc.) – exclusivo para profesores y administrativos. Sistema de alertas programadas (deadlines unificados). Auditoría en tiempo real visible para supervisores.	Azure Logic Apps, librerías de gráficos (Chart.js, D3), mejoras frontend
Fase 4 - Optimización y Cierre	Mes 6	Pruebas de carga, optimización de consultas, refinamiento de la interfaz para estudiantes (solo texto, sin gráficos). Documentación final y entrega.	Azure Performance Testing, pulido frontend

6. Beneficios de la Arquitectura Ajustada

- **Autenticación simplificada y segura**

  JWT manejado íntegramente por FastAPI, sin dependencias de servicios externos de identidad.

- **Auditoría completa y trazable**

  Cada cambio sobre cualquier entidad queda registrado con usuario, timestamp y valores previos, cumpliendo con los principios de calidad del dato.

- **Alertas unificadas por deadlines**

  Un solo mecanismo cubre tanto proyectos próximos a vencer como semilleros inactivos, reduciendo complejidad operativa.

- **Gráficos exclusivos y adaptados por rol**

Profesores y administrativos ven dashboards visuales adaptados a sus necesidades; los estudiantes no acceden a gráficos, simplificando la interfaz y los permisos.

- **Modularidad y escalabilidad en Azure**

  Los componentes (Base de datos, Storage, Logic Apps) son servicios PaaS que escalan bajo demanda y permiten incorporar nuevas capacidades sin reestructurar.

- **Costo eficiente y plazo ajustado**

  Uso de servicios con pago por consumo y un roadmap de 6 meses que entrega un MVP completo con gráficos, alertas y auditoría.

- **Experiencia de usuario diferenciada**

  Tres perfiles claros: administrativo (gráficos agregados), profesor (gráficos de sus proyectos), estudiante (solo formularios y consultas de texto).